

Code:

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <ctype.h>
5
6 #define MAX_TOKENS 100
7 #define MAX_LEN 100
8
9 // Token types
10 typedef enum {
11     COMMAND,
12     ARGUMENT,
13     PIPE,
14     REDIRECT_IN,
15     REDIRECT_OUT,
16     SEMICOLON
17 } TokenType;
18
19 // Token structure
20 typedef struct {
21     char value[MAX_LEN];
22     TokenType type;
23 } Token;
24
25 // Convert token type to string
26 const char* getTokenType(TokenType type) {
27     switch (type) {
28         case COMMAND: return "COMMAND";
29         case ARGUMENT: return "ARGUMENT";
30         case PIPE: return "PIPE";
31         case REDIRECT_IN: return "REDIRECT_IN";
32         case REDIRECT_OUT: return "REDIRECT_OUT";
33         case SEMICOLON: return "SEMICOLON";
34         default: return "UNKNOWN";
35     }
36 }
37
38 // Check if character is a delimiter
39 int isDelimiter(char ch) {
40     return ch == '|' || ch == '<' || ch == '>' || ch == ';';
41 }
42
43 // Tokenize input
44 int tokenize(char *input, Token tokens[]) {
45     int count = 0;
46     int i = 0;
47     int firstWord = 1;
48
49     while (input[i] != '\0') {
50
51         // Handle whitespace
52         if (isspace(input[i])) {
53             i++;
54             continue;
55         }
```

```

56
57 // Handle delimiters
58 if (isDelimiter(input[i])) {
59
60     tokens[count].value[0] = input[i];
61     tokens[count].value[1] = '\0';
62
63     if (input[i] == '|')
64         tokens[count].type = PIPE;
65     else if (input[i] == '<')
66         tokens[count].type = REDIRECT_IN;
67     else if (input[i] == '>')
68         tokens[count].type = REDIRECT_OUT;
69     else
70         tokens[count].type = SEMICOLON;
71
72     count++;
73     i++;
74     firstWord = 1;
75     continue;
76 }
77
78 // Read normal word
79 int j = 0;
80
81 while (input[i] != '\0' &&
82        !isspace(input[i]) &&
83        !isDelimiter(input[i])) {
84
85     if (j < MAX_LEN - 1)
86         tokens[count].value[j++] = input[i];
87
88     i++;
89 }
90
91 tokens[count].value[j] = '\0';
92
93 // First word after delimiter is command
94 if (firstWord) {
95     tokens[count].type = COMMAND;
96     firstWord = 0;
97 } else {
98     tokens[count].type = ARGUMENT;
99 }
100
101 count++;
102 }
103
104 return count;
105 }
106
107 // Validate token stream
108 int validateTokens(Token tokens[], int count) {

```

```

109
110     if (count == 0) {
111         printf("Error: Empty command.\n");
112         return 0;
113     }
114
115     // Command cannot start with pipe
116     if (tokens[0].type == PIPE) {
117         printf("Error: Command cannot start with '|'.\n");
118         return 0;
119     }
120
121     // Command cannot end with pipe
122     if (tokens[count - 1].type == PIPE) {
123         printf("Error: Command cannot end with '|'.\n");
124         return 0;
125     }
126
127     // Two consecutive pipes are invalid
128     for (int i = 0; i < count - 1; i++) {
129         if (tokens[i].type == PIPE &&
130             tokens[i + 1].type == PIPE) {
131
132             printf("Error: Consecutive pipes are not allowed.\n");
133             return 0;
134         }
135     }
136
137     return 1;
138 }
139
140 // Display tokens for debugging
141 void debugTokens(Token tokens[], int count) {
142
143     printf("\n--- TOKEN STREAM ---\n");
144
145     for (int i = 0; i < count; i++) {
146         printf("Token %d: %-10s Type: %s\n",
147             i + 1,
148             tokens[i].value,
149             getTokenType(tokens[i].type));
150     }
151 }
152
153 // Generate simple parse/execution structure
154 void generateExecutionStructure(Token tokens[], int count) {
155
156     printf("\n--- EXECUTION STRUCTURE ---\n");
157
158     printf("Command: ");
159
160     for (int i = 0; i < count; i++) {
161

```

```

162         if (tokens[i].type == PIPE) {
163             printf("\n      |\n      v\nCommand: ");
164         }
165         else if (tokens[i].type == REDIRECT_IN) {
166             printf(" [INPUT REDIRECTION] ");
167         }
168         else if (tokens[i].type == REDIRECT_OUT) {
169             printf(" [OUTPUT REDIRECTION] ");
170         }
171         else if (tokens[i].type == SEMICOLON) {
172             printf("\n      ;\nCommand: ");
173         }
174         else {
175             printf("%s ", tokens[i].value);
176         }
177     }
178
179     printf("\n");
180 }
181
182 // Main function
183 int main() {
184
185     char input[500];
186     Token tokens[MAX_TOKENS];
187
188     printf("Enter command: ");
189     fgets(input, sizeof(input), stdin);
190
191     // Remove newline
192     input[strcspn(input, "\n")] = '\0';
193
194     // Tokenization
195     int count = tokenize(input, tokens);
196
197     // Debug token output
198     debugTokens(tokens, count);
199
200     // Validate syntax
201     if (!validateTokens(tokens, count)) {
202         return 1;
203     }
204
205     printf("\nSyntax validation successful.\n");
206
207     // Generate execution structure
208     generateExecutionStructure(tokens, count);
209
210     return 0;
211 }
212

```

Output:

```

saisurendra@SaiSurendra:~$ nano Skill4.c
saisurendra@SaiSurendra:~$ gcc Skill4.c -o Skill4
saisurendra@SaiSurendra:~$ ./Skill4
Enter command: ls

--- TOKEN STREAM ---
Token 1: ls          Type: COMMAND

Syntax validation successful.

--- EXECUTION STRUCTURE ---
Command: ls

```